

Normal Forms in DBMS

Last Updated : 20 Nov, 2025

-
-
-

Normal forms are a set of progressive rules (or design checkpoints) for relational schemas that reduce redundancy and prevent data anomalies. Each normal form - 1NF, 2NF, 3NF, BCNF, 4NF, 5NF - is stricter than the previous one: meeting a higher normal form implies the lower ones are satisfied. Think of them as layers of cleanliness for your tables: the deeper you go, the fewer redundancy and integrity problems you'll have.

Benefits of using Normal Forms

- Reduce duplicate data and wasted storage.
- Prevent insert, update, and delete anomalies.
- Improve data consistency and integrity.
- Make the schema easier to maintain and evolve.

The Diagram below shows the hierarchy of database normal forms. Each inner circle represents a stricter level of normalization, starting from 1NF (basic structure) to 5NF (most refined). As you move inward, data redundancy reduces and data integrity improves. Each level builds upon the previous one to ensure a cleaner and more efficient database design.

Let's break down the various normal forms step-by-step to understand the conditions that need to be satisfied at each level:

1. First Normal Form (1NF): Eliminating Duplicate Records

A table is in [1NF](#) if it satisfies the following conditions:

- All columns contain atomic values (i.e., indivisible values).
- Each row is unique (i.e., no duplicate rows).
- Each column has a unique name.
- The order in which data is stored does not matter.

Example of 1NF Violation: If a table has a column "Phone Numbers" that stores multiple phone numbers in a single cell, it violates 1NF. To bring it into 1NF, you need to separate phone numbers into individual rows.

2. Second Normal Form (2NF): Eliminating Partial Dependency

A relation is in [2NF](#) if it satisfies the conditions of 1NF and additionally. No partial dependency exists, meaning every non-prime attribute (non-key attribute) must depend on the entire primary key, not just a part of it.

Example: For a composite key (StudentID, CourseID), if the "StudentName" depends only on "StudentID" and not on the entire key, it violates 2NF. To normalize, move StudentName into a separate table where it depends only on "StudentID".

3. Third Normal Form (3NF): Eliminating Transitive Dependency

A relation is in [3NF](#) if it satisfies 2NF and additionally, there are no [transitive dependencies](#). In simpler terms, non-prime attributes should not depend on other non-prime attributes.

Example: Consider a table with (StudentID, CourseID, Instructor). If Instructor depends on "CourseID", and "CourseID" depends on "StudentID", then Instructor indirectly depends on "StudentID", which violates 3NF. To resolve this, place Instructor in a separate table linked by "CourseID".

4. Boyce-Codd Normal Form (BCNF): The Strongest Form of 3NF

[BCNF](#) is a stricter version of 3NF where for every non-trivial functional dependency ($X \rightarrow Y$), X must be a [superkey](#) (a unique identifier for a record in the table).

Example: If a table has a dependency (StudentID, CourseID) $\rightarrow$ Instructor, but neither "StudentID" nor "CourseID" is a superkey, then it violates BCNF. To bring it into BCNF, decompose the table so that each determinant is a candidate key.

5. Fourth Normal Form (4NF): Removing Multi-Valued Dependencies

A table is in [4NF](#) if it is in BCNF and has no multi-valued dependencies. A multi-valued dependency occurs when one attribute determines another, and both attributes are independent of all other attributes in the table.

Example: Consider a table where (StudentID, Language, Hobby) are attributes. If a student can have multiple hobbies and languages, a multi-valued dependency exists. To resolve this, split the table into separate tables for Languages and Hobbies.

6. Fifth Normal Form (5NF): Eliminating Join Dependency

[5NF](#) is achieved when a table is in 4NF and all join dependencies are removed. This form ensures that every table is fully decomposed into smaller tables that are logically connected without losing information.

Example: If a table contains (StudentID, Course, Instructor) and there is a dependency where all combinations of these columns are needed for a specific relationship, you would split them into smaller tables to remove redundancy.